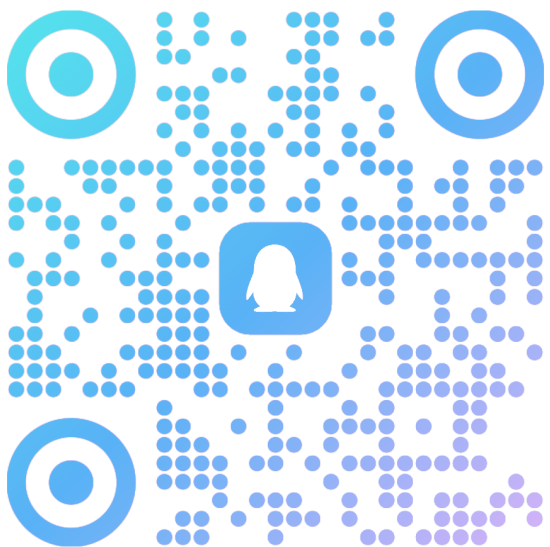




福州大学-2025编译...

群号: 1037351079



扫一扫二维码，加入群聊



编译原理

2025 春

任课教师: 柯嵩宇

2025-03-06

课前复习

编程语言的构成

语法

- 声明
- 定义
- 语句

语义

- 顺序、分支、循环
- 面向对象
- 函数式编程
- 类型转换

语言标准库

- 输入输出函数
- 访问文件的接口
- 内存申请、释放
- 常用数据结构,
list、*map*
- 信息安全相关函数
- 网络访问支持

四类文法

Grammar ⇄	Languages ⇄	Recognizing automaton ⇄	Production rules (constraints) ^[a] ⇄	Examples ^{[5][6]} ⇄
Type-3	Regular	Finite-state automaton	$A \rightarrow a$ $A \rightarrow aB$ (right regular) or $A \rightarrow a$ $A \rightarrow Ba$ (left regular)	$L = \{a^n n > 0\}$
Type-2	Context-free	Non-deterministic pushdown automaton	$A \rightarrow \alpha$	$L = \{a^n b^n n > 0\}$
Type-1	Context-sensitive	Linear-bounded non-deterministic Turing machine	$\alpha A \beta \rightarrow \alpha \gamma \beta$	$L = \{a^n b^n c^n n > 0\}$
Type-0	Recursively enumerable	Turing machine		$L = \{w w \text{ describes a halting Turing machine}\}$

绝大多数编程语言的语法都是上下文无关文法，但如果要考虑语义信息的话，它就变成了上下文相关语言了

有穷状态自动机 (Finite Automation)

- 有穷状态自动机，也称有限状态自动机，可以被定义成一个五元组 $(\Sigma, S, s_0, \delta, F)$
 - Σ 是字符集(alphabet)
 - S 是状态集合(states)
 - s_0 是初始状态(initial state)
 - δ 是状态转移函数(transition function)
 $\delta(s, \sigma) = t \in S$
 - F 是可接受状态(accepting state)的集合。
 - 可接受状态 $f \in S$ ，是一种特殊的状态。可以大致理解为，如果自动机处理完了所有输入后停留在可接受状态，它就会返回True

非确定有穷状态自动机

(Nondeterministic Finite Automation)

- 非确定有穷状态自动机可以被定义成一个五元组

$$(\Sigma, S, s_0, \delta, F)$$

- Σ 是字符集(alphabet)
- S 是状态集合(states)
- s_0 是初始状态(initial state)
- δ 是状态转移函数(transition function)
- F 是可接受状态(accepting state)的集合。
- 状态转移函数 δ 的定义如下

$$\delta(s, \sigma) = \{t_1, t_2, \dots\} \subseteq S$$

- 特别地, σ 允许为 ε

正则语言与正则表达式

正则语言

- 令 r 为一个正则表达式, 则 $L(r)$ 为其所定义的正则语言

正则表达式的归纳定义

- 空串 ε 是一个正则表达式，对应的正则语言 $L(\varepsilon) = \Phi$
- 字符集中的每个独立终结符号都是一个正则表达式，其所定义的正则语言就是只包含该字符的集合

$$L(a) = \{a\}, L(b) = \{b\}$$

- 两个正则表达式拼接之后还是一个正则表达式

$$L(r_1 r_2) = \{s_1 s_2 \mid s_1 \in L(r_1) \wedge s_2 \in L(r_2)\}$$

- 两个语言的并——正则表达式的或

$$L(r_1 + r_2) = L(r_1) \cup L(r_2)$$

用“ $|$ ”表示“或”是IEEE的标准，“ $+$ ”是大学在教学时经常使用的，因为这个操作符合我们对加法朴素的印象——结合律、交换律、有“零”元 (Φ)

有必要的扩展

- 正则表达式的幂

$$r^0 = \varepsilon, r^1 = r, r^2 = rr$$

- Kleene Star(闭包)

$$r^* = \sum_{i=0}^{\infty} r^i$$

$$L(r^*) = \bigcup_{i=0}^{\infty} L(r^i)$$

有必要的扩展

- Kleene Plus(正闭包, positive closure)

$$r^+ = \sum_{i=1}^n r^i$$

$$L(r^+) = \bigcup_{i=1}^{\infty} L(r^i)$$

有必要的扩展

- 可选闭包(?)

$$r? = r + \varepsilon$$

$$L(r?) = L(r) \cup \Phi$$

运算的优先级

- 与一般的数学运算相同
 - 幂最高，作用于它左边的第一个操作数
 - 其次是乘法
 - 再次是加法
- 括号改变运算优先级

课堂练习

- 写一个01字符的正则表达式，其所定义的语言不包含“100”子串

课堂练习

- 写一个01字符的正则表达式，其所定义的语言不包含“100”子序列

课堂练习

- 写一个01正则表达式，其所定义的语言仅包含所有由偶数个1和奇数个0构成的串

正则表达式、NFA、DFA

从正则表达式到NFA

- 最小化原理
- 拼接
- 并

例题

- 写一个01字符的正则表达式，其所定义的语言不包含“100”子串
- 构建该RE对应的

课堂练习

- 写一个01字符的正则表达式，其所定义的语言不包含“100”子序列

课堂练习

- 写一个01正则表达式，其所定义的语言仅包含所有由偶数个1和奇数个0构成的串

重点： 从NFA到DFA

非确定有穷状态自动机

(Nondeterministic Finite Automation)

- 非确定有穷状态自动机可以被定义成一个五元组

$$(\Sigma, S, s_0, \delta, F)$$

- Σ 是字符集(alphabet)
- S 是状态集合(states)
- s_0 是初始状态(initial state)
- δ 是状态转移函数(transition function)
- F 是可接受状态(accepting state)的集合。
- 状态转移函数 δ 的定义如下

$$\delta(s, \sigma) = \{t_1, t_2, \dots\} \subseteq S$$

- 特别地, σ 允许为 ε

有穷状态自动机 (Finite Automation)

- 有穷状态自动机，也称有限状态自动机，可以被定义成一个五元组

$$(\Sigma, S, s_0, \delta, F)$$

- Σ 是字符集(alphabet)
- S 是状态集合(states)
- s_0 是初始状态(initial state)
- δ 是状态转移函数(transition function)
 $\delta(s, \sigma) = t \in S$
- F 是可接受状态(accepting state)的集合

重点：从NFA到DFA

- 核心思想
 - 用NFA所有可能状态的集合作为DFA的状态
 - 例： $\{s_1, s_2, s_4\}$, $\{s_4, s_6\}$, ...
- 具体步骤
 1. 确定起始状态
 - NFA的起始状态和所有通过 ϵ 转移能够到达的状态
 2. 构建DFA的状态转移表
 - 对每个DFA状态集合和输入符号，计算转移后的新状态
 - 计算集合状态中每个子状态在输入 a 下的转移状态集合，对所有子状态的转移集合取并集成为新的状态集合
 - 如果新的状态集合是从未出现过的，那么放入队列以后续处理
 3. 确定DFA的接受状态
 - 包含NFA任意接受状态的集合即为DFA接受状态

课堂练习

- 写一个01字符的正则表达式，其所定义的语言不包含“100”子串

课堂练习

- 写一个01字符的正则表达式，其所定义的语言不包含“100”子序列

课堂练习

- 写一个01正则表达式，其所定义的语言仅包含所有由偶数个1和奇数个0构成的串